USAGE AND COSTS

Monitoring

Learn how to enable and configure OpenTelemetry for Claude Code.

Track Claude Code usage, costs, and tool activity across your organization by exporting telemetry data through OpenTelemetry (OTel). Claude Code exports metrics as time series data via the standard metrics protocol, events via the logs/events protocol, and optionally distributed traces via the **traces protocol**. Configure your metrics, logs, and traces backends to match your monitoring requirements.

Quick start

Configure OpenTelemetry using environment variables:

```
# 1. Enable telemetry
export CLAUDE_CODE_ENABLE_TELEMETRY=1

# 2. Choose exporters (both are optional - configure only what
you need)
export OTEL_METRICS_EXPORTER=otlp       # Options: otlp,
prometheus, console, none
export OTEL_LOGS_EXPORTER=otlp          # Options: otlp, console,
none

# 3. Configure OTLP endpoint (for OTLP exporter)
export OTEL_EXPORTER_OTLP_PROTOCOL=grpc
export OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317

# 4. Set authentication (if required)
export OTEL_EXPORTER_OTLP_HEADERS="Authorization=Bearer your-
token"

# 5. For debugging: reduce export intervals
export OTEL_METRIC_EXPORT_INTERVAL=10000  # 10 seconds (default:
60000ms)
export OTEL_LOGS_EXPORT_INTERVAL=5000     # 5 seconds (default:
5000ms)

# 6. Run Claude Code
claude
```

The default export intervals are 60 seconds for metrics and 5 seconds for logs. During setup, you may want to use shorter intervals for debugging purposes. Remember to reset these for production use.

For full configuration options, see the **OpenTelemetry specification**.

Administrator configuration

Administrators can configure OpenTelemetry settings for all users through the **managed settings file**. This allows for centralized control of telemetry settings across an organization. See the **settings precedence** for more information about how settings are applied.

Example managed settings configuration:

```
{
  "env": {
    "CLAUDE_CODE_ENABLE_TELEMETRY": "1",
    "OTEL_METRICS_EXPORTER": "otlp",
    "OTEL_LOGS_EXPORTER": "otlp",
    "OTEL_EXPORTER_OTLP_PROTOCOL": "grpc",
    "OTEL_EXPORTER_OTLP_ENDPOINT":
"http://collector.example.com:4317",
    "OTEL_EXPORTER_OTLP_HEADERS": "Authorization=Bearer example-
token"
  }
}
```

Managed settings can be distributed via MDM (Mobile Device Management) or other device management solutions. Environment variables defined in the managed settings file have high precedence and can't be overridden by users.

Claude Code doesn't pass `OTEL_*` environment variables to the subprocesses it spawns, including the Bash tool, hooks, MCP servers, and language servers. An OpenTelemetry-instrumented application that you run through the Bash tool doesn't inherit Claude Code's exporter endpoint or headers, so set those variables directly in the command if that application needs to export its own telemetry.

Configuration details

Common configuration variables

Environment Variable	Description	Example Values
`CLAUDE_CODE_ENABLE_TELEMETRY`	Enables telemetry collection (required)	`1`
`OTEL_METRICS_EXPORTER`	Metrics exporter types, comma-separated. Use `none` to disable	`console`, `otlp`, `prometheus`, `none`
`OTEL_LOGS_EXPORTER`	Logs/events exporter types, comma-separated. Use `none` to disable	`console`, `otlp`, `none`
`OTEL_EXPORTER_OTLP_PROTOCOL`	Protocol for OTLP exporter, applies to all signals	`grpc`, `http/json`, `http/protobuf`
`OTEL_EXPORTER_OTLP_ENDPOINT`	OTLP collector endpoint for all signals	`http://localhost:4317`

Environment Variable	Description	Example Values
`OTEL_EXPORTER_OTLP_METRICS_PROTOCOL`	Protocol for metrics, overrides general setting	`grpc`, `http/json`, `http/protobuf`
`OTEL_EXPORTER_OTLP_METRICS_ENDPOINT`	OTLP metrics endpoint, overrides general setting	`http://localhost:4318/v1/metrics`
`OTEL_EXPORTER_OTLP_LOGS_PROTOCOL`	Protocol for logs, overrides general setting	`grpc`, `http/json`, `http/protobuf`
`OTEL_EXPORTER_OTLP_LOGS_ENDPOINT`	OTLP logs endpoint, overrides general setting	`http://localhost:4318/v1/logs`
`OTEL_EXPORTER_OTLP_HEADERS`	Authentication headers for OTLP	`Authorization=Bearer token`
`OTEL_METRIC_EXPORT_INTERVAL`	Export interval in milliseconds (default: 60000)	`5000`, `60000`
`OTEL_LOGS_EXPORT_INTERVAL`	Logs export interval in milliseconds (default: 5000)	`1000`, `10000`
`OTEL_LOG_USER_PROMPTS`	Enable logging of user prompt content (default: disabled)	`1` to enable
`OTEL_LOG_TOOL_DETAILS`	Enable logging of tool parameters and input arguments in tool events and trace span attributes: Bash commands, MCP server and tool names, skill names, and tool input. Also enables custom, plugin, and MCP command names on `user_prompt` events (default: disabled)	`1` to enable
`OTEL_LOG_TOOL_CONTENT`	Enable logging of tool input and output content in span events (default: disabled). Requires **tracing**. Content is truncated at 60 KB	`1` to enable
`OTEL_LOG_RAW_API_BODIES`	Emit the full Anthropic Messages API request and response JSON as `api_request_body` / `api_response_body` log events (default: disabled). Bodies include the entire conversation history. Enabling this implies consent to everything `OTEL_LOG_USER_PROMPTS`, `OTEL_LOG_TOOL_DETAILS`, and `OTEL_LOG_TOOL_CONTENT` would reveal	`1` for inline bodies truncated at 60 KB, or `file:<dir>` for untruncated bodies on disk with a `body_ref` pointer in the event
`OTEL_EXPORTER_OTLP_METRICS_TEMPORALITY_PREFERENCE`	Metrics temporality preference (default: `delta`). Set to `cumulative` if your backend expects cumulative temporality	`delta`, `cumulative`
`CLAUDE_CODE_OTEL_HEADERS_HELPER_DEBOUNCE_MS`	Interval for refreshing dynamic headers (default: 1740000ms / 29 minutes)	`900000`

mTLS authentication

How you configure client certificates for the OTLP exporter depends on the OTLP protocol in use for that signal, set via `OTEL_EXPORTER_OTLP_PROTOCOL` or the per-signal override. The same configuration applies to metrics, logs, and traces.

Protocol	Client certificate variables	Trust the collector's CA with
`http/protobuf`, `http/json`	`CLAUDE_CODE_CLIENT_CERT`, `CLAUDE_CODE_CLIENT_KEY`, and optionally `CLAUDE_CODE_CLIENT_KEY_PASSPHRASE`. See **Network configuration**	`NODE_EXTRA_CA_CERTS`
`grpc`	`OTEL_EXPORTER_OTLP_CLIENT_KEY` and `OTEL_EXPORTER_OTLP_CLIENT_CERTIFICATE`, or the per-signal variants such as `OTEL_EXPORTER_OTLP_METRICS_CLIENT_KEY` to use a different certificate per signal	`OTEL_EXPORTER_OTLP_CERTIFICATE`

For `grpc`, the OpenTelemetry SDK reads the standard OTLP variables directly, so existing configurations that set the per-signal metrics variables continue to work.

Metrics cardinality control

The following environment variables control which attributes are included in metrics to manage cardinality:

Environment Variable	Description	Default Value	Example to Disable
`OTEL_METRICS_INCLUDE_SESSION_ID`	Include session.id attribute in metrics	`true`	`false`
`OTEL_METRICS_INCLUDE_VERSION`	Include app.version attribute in metrics	`false`	`true`
`OTEL_METRICS_INCLUDE_ACCOUNT_UUID`	Include user.account_uuid and user.account_id attributes in metrics	`true`	`false`
`OTEL_METRICS_INCLUDE_ENTRYPOINT`	Include app.entrypoint attribute in metrics	`false`	`true`
`OTEL_METRICS_INCLUDE_RESOURCE_ATTRIBUTES`	Include keys from `OTEL_RESOURCE_ATTRIBUTES` as attributes on metric datapoints	`true`	`false`

These variables help control the cardinality of metrics, which affects storage requirements and query performance in your metrics backend. Lower cardinality generally means better performance

and lower storage costs but less granular data for analysis.

Traces (beta)

Distributed tracing exports spans that link each user prompt to the API requests and tool executions it triggers, so you can view a full request as a single trace in your tracing backend.

Tracing is off by default. To enable it, set both `CLAUDE_CODE_ENABLE_TELEMETRY=1` and `CLAUDE_CODE_ENHANCED_TELEMETRY_BETA=1`, then set `OTEL_TRACES_EXPORTER` to choose where spans are sent. Traces reuse the **common OTLP configuration** for endpoint, protocol, headers, and **mTLS**.

Environment Variable	Description	Example Values
`CLAUDE_CODE_ENHANCED_TELEMETRY_BETA`	Enable span tracing (required). `ENABLE_ENHANCED_TELEMETRY_BETA` is also accepted	`1`
`OTEL_TRACES_EXPORTER`	Traces exporter types, comma-separated. Use `none` to disable	`console`, `otlp`, `none`
`OTEL_EXPORTER_OTLP_TRACES_PROTOCOL`	Protocol for traces, overrides `OTEL_EXPORTER_OTLP_PROTOCOL`	`grpc`, `http/json`, `http/protobuf`
`OTEL_EXPORTER_OTLP_TRACES_ENDPOINT`	OTLP traces endpoint, overrides `OTEL_EXPORTER_OTLP_ENDPOINT`	`http://localhost:4318/v1/traces`
`OTEL_TRACES_EXPORT_INTERVAL`	Span batch export interval in milliseconds (default: 5000)	`1000`, `10000`

Spans redact user prompt text, tool input details, and tool content by default. Set `OTEL_LOG_USER_PROMPTS=1`, `OTEL_LOG_TOOL_DETAILS=1`, and `OTEL_LOG_TOOL_CONTENT=1` to include them.

When tracing is active, Bash and PowerShell subprocesses automatically inherit a `TRACEPARENT` environment variable containing the W3C trace context of the active tool execution span. This lets any subprocess that reads `TRACEPARENT` parent its own spans under the same trace, enabling end-to-end distributed tracing through scripts and commands that Claude runs.

When tracing is active and Claude Code is connected directly to the Anthropic API, each model request carries a W3C `traceparent` header set to the `claude_code.llm_request` span's context, and the API's `traceresponse` header is recorded as a span link. Together these connect Claude Code's client-side spans to the server-side trace through any compliant intermediary. Outbound HTTP MCP requests carry `traceparent` the same way. The header is not sent to third-party

providers.

By default, the `traceparent` header on model and HTTP MCP requests is sent only when `ANTHROPIC_BASE_URL` is unset or points at the Anthropic API, since some proxies reject unrecognized headers. The subprocess `TRACEPARENT` variable is controlled by the same switch for consistency. If you run Claude Code through a custom `ANTHROPIC_BASE_URL` proxy and want trace context propagated, set `CLAUDE_CODE_PROPAGATE_TRACEPARENT=1`.

In Agent SDK and non-interactive sessions started with `-p`, Claude Code also reads `TRACEPARENT` and `TRACESTATE` from its own environment when starting each interaction span. This lets an embedding process pass its active W3C trace context into the subprocess so Claude Code's spans appear as children of the caller's distributed trace. Interactive sessions ignore inbound `TRACEPARENT` to avoid accidentally inheriting ambient values from CI or container environments.

Span hierarchy

Each user prompt starts a `claude_code.interaction` root span. API calls, tool calls, and hook executions are recorded as its children. Tool spans have two child spans of their own: one for the time spent waiting on a permission decision and one for the execution itself. When the Agent tool, or legacy Task tool, spawns a subagent, the subagent's API and tool spans nest under the parent's `claude_code.tool` span.

```
claude_code.interaction
├── claude_code.llm_request
├── claude_code.hook                    (requires detailed beta
tracing)
└── claude_code.tool
    ├── claude_code.tool.blocked_on_user
    ├── claude_code.tool.execution
    └── (Agent tool) subagent claude_code.llm_request /
claude_code.tool spans
```

In Agent SDK and `claude -p` sessions, `claude_code.interaction` itself becomes a child of the caller's span when `TRACEPARENT` is set in the environment.

Span attributes

Every span carries the **standard attributes** plus a `span.type` attribute matching its name. The tables below list the additional attributes set on each span. The `llm_request`, `tool.execution`, and `hook` spans set OpenTelemetry status `ERROR` when they record a failure; the other spans

always end with status `UNSET`.

`claude_code.interaction`

Attribute	Description	Gated by
`user_prompt`	Prompt text. Value is `<REDACTED>` unless the gate is set	`OTEL_LOG_USER_PROMPTS`
`user_prompt_length`	Prompt length in characters	
`interaction.sequence`	1-based counter of interactions in this session	
`interaction.duration_ms`	Wall-clock duration of the turn	

`claude_code.llm_request`

Attribute	Description	Gated by
`model`	Model identifier	
`gen_ai.system`	Always `anthropic`. OpenTelemetry GenAI semantic convention	
`gen_ai.request.model`	Same value as `model`. OpenTelemetry GenAI semantic convention	
`query_source`	Subsystem that issued the request, such as `repl_main_thread` or a subagent name	
`agent_id`	Identifier of the subagent or teammate that issued the request. Absent on the main session	
`parent_agent_id`	Identifier of the agent that spawned this one. Absent for the main session and for agents spawned directly from it	
`speed`	`fast` or `normal`	
`llm_request.context`	`interaction`, `tool`, or `standalone` depending on the parent span	
`duration_ms`	Wall-clock duration including retries	
`ttft_ms`	Time to first token in milliseconds	
`input_tokens`	Input token count from the API usage block	
`output_tokens`	Output token count	
`cache_read_tokens`	Tokens read from prompt cache	
`cache_creation_tokens`	Tokens written to prompt cache	

Attribute	Description	Gated by
`request_id`	Anthropic API request ID from the `request-id` response header	
`gen_ai.response.id`	Same value as `request_id`. OpenTelemetry GenAI semantic convention	
`client_request_id`	Client-generated `x-client-request-id` of the final attempt	
`attempt`	Total attempts made for this request	
`success`	`true` or `false`	
`status_code`	HTTP status code when the request failed	
`error`	Error message when the request failed	
`response.has_tool_call`	`true` when the response contained tool-use blocks	
`stop_reason`	API response `stop_reason`, such as `end_turn`, `tool_use`, `max_tokens`, `stop_sequence`, `pause_turn`, or `refusal`	
`gen_ai.response.finish_reasons`	Same value as `stop_reason`, wrapped in a string array. OpenTelemetry GenAI semantic convention	

Each retry attempt is also recorded as a `gen_ai.request.attempt` span event with `attempt` and `client_request_id` attributes.

`claude_code.tool`

Attribute	Description	Gated by
`tool_name`	Tool name	
`duration_ms`	Wall-clock duration including permission wait and execution	
`result_tokens`	Approximate token size of the tool result	
`agent_id`	Identifier of the subagent or teammate that ran the tool. Absent on the main session	
`parent_agent_id`	Identifier of the agent that spawned this one. Absent for the main session and for agents spawned directly from it	
`tool_use_id`	The model's `tool_use` block id for this call. Matches the `tool_use_id` on the **tool_result** and **tool_decision** events and in hook payloads, so you can join the span to those records	
`gen_ai.tool.call.id`	Same value as `tool_use_id`. OpenTelemetry GenAI semantic convention	

Attribute	Description	Gated by
`file_path`	Target file path for Read, Edit, and Write tools	`OTEL_LOG_TOOL_DETAILS`
`full_command`	Command string for the Bash tool	`OTEL_LOG_TOOL_DETAILS`
`skill_name`	Skill name for the Skill tool	`OTEL_LOG_TOOL_DETAILS`
`subagent_type`	Subagent type for the Agent tool or legacy Task tool	`OTEL_LOG_TOOL_DETAILS`

When `OTEL_LOG_TOOL_CONTENT=1`, this span also records a `tool.output` span event whose attributes contain the tool's input and output bodies, truncated at 60 KB per attribute.

`claude_code.tool.blocked_on_user`

Attribute	Description	Gated by
`duration_ms`	Time spent waiting for the permission decision	
`decision`	`accept` or `reject`	
`source`	Decision source, matching the **Tool decision event**	

`claude_code.tool.execution`

Attribute	Description	Gated by
`duration_ms`	Time spent running the tool body	
`tool_use_id`	Same value as on the parent `claude_code.tool` span	
`gen_ai.tool.call.id`	Same value as `tool_use_id`. OpenTelemetry GenAI semantic convention	
`success`	`true` or `false`	
`error`	Error category string when execution failed, such as `Error:ENOENT` or `ShellError`. Contains the full error message instead when the gate is set	`OTEL_LOG_TOOL_DETAILS`

`claude_code.hook`

This span is emitted only when detailed beta tracing is active, which requires `ENABLE_BETA_TRACING_DETAILED=1` and `BETA_TRACING_ENDPOINT` in addition to the trace exporter

configuration above. In interactive CLI sessions, this also requires your organization to be allowlisted for the feature. Agent SDK and non-interactive `-p` sessions are not gated. It is not emitted when only `CLAUDE_CODE_ENHANCED_TELEMETRY_BETA` is set.

Attribute	Description	Gated by
`hook_event`	Hook event type, such as `PreToolUse`	
`hook_name`	Full hook name, such as `PreToolUse:Write`	
`num_hooks`	Number of matching hook commands executed	
`hook_definitions`	JSON-serialized hook configuration	`OTEL_LOG_TOOL_DETAILS`
`duration_ms`	Wall-clock duration of all matching hooks	
`num_success`	Count of hooks that completed successfully	
`num_blocking`	Count of hooks that returned a blocking decision	
`num_non_blocking_error`	Count of hooks that failed without blocking	
`num_cancelled`	Count of hooks cancelled before completion	

Additional content-bearing attributes such as `new_context`, `system_prompt_preview`, `user_system_prompt`, `tool_input`, and `response.model_output` are emitted only when detailed beta tracing is active. They are not part of the stable span schema. `user_system_prompt` additionally requires `OTEL_LOG_USER_PROMPTS=1`. It carries only the system prompt text you provide via the `systemPrompt` SDK option or `--system-prompt` and `--append-system-prompt` flags, truncated at 60 KB, and is emitted once per session rather than per request.

Dynamic headers

For enterprise environments that require dynamic authentication, you can configure a script to generate headers dynamically. Dynamic headers apply only to the `http/protobuf` and `http/json` protocols. The `grpc` exporter uses only the static `OTEL_EXPORTER_OTLP_HEADERS` value.

Settings configuration

Add to your `.claude/settings.json`:

```
{
  "otelHeadersHelper": "/bin/generate_opentelemetry_headers.sh"
}
```

The value can be the path to an executable file, including a path that contains spaces, or a shell command line with arguments. On Windows, the value always runs through the shell, so quote a path that contains spaces inside the JSON value.

Script requirements

The script must output valid JSON with string key-value pairs representing HTTP headers:

```
#!/bin/bash
# Example: Multiple headers
echo "{\"Authorization\": \"Bearer $(get-token.sh)\", \"X-API-
Key\": \"$(get-api-key.sh)\"}"
```

If the helper fails or prints output that doesn't meet these requirements, Claude Code reports the error in:

- `/doctor` output
- The debug log, when running with `--debug` or after running `/debug` in the session
- stderr, in non-interactive sessions started with `-p`

Refresh behavior

The headers helper script runs at startup and periodically thereafter to support token refresh. By default, the script runs every 29 minutes. Customize the interval with the `CLAUDE_CODE_OTEL_HEADERS_HELPER_DEBOUNCE_MS` environment variable.

Multi-team organization support

Organizations with multiple teams or departments can add custom attributes to distinguish between different groups using the `OTEL_RESOURCE_ATTRIBUTES` environment variable:

```
# Add custom attributes for team identification
export
OTEL_RESOURCE_ATTRIBUTES="department=engineering,team.id=platform
,cost_center=eng-123"
```

These custom attributes are included in all metrics and events, allowing you to:

- Filter metrics by team or department
- Track costs per cost center
- Create team-specific dashboards
- Set up alerts for specific teams

Claude Code attaches these values as attributes on every metric datapoint and event record, in addition to sending them in the OTLP resource block. Because most metrics backends expose datapoint attributes as queryable labels, you can group and filter metrics by your custom keys directly. Custom keys never override the **standard attributes** such as `user.id` or `session.id`: when a key collides, Claude Code keeps the built-in value.

Each custom key becomes a label on every metric series, so high-cardinality values increase storage cost in your metrics backend. To send custom attributes in the resource block only and omit them from datapoint labels, set `OTEL_METRICS_INCLUDE_RESOURCE_ATTRIBUTES=false`. See **Metrics cardinality control**.

⚠ The `OTEL_RESOURCE_ATTRIBUTES` environment variable uses comma-separated key=value pairs with strict formatting requirements:

- **No spaces allowed**: values can't contain spaces. For example, `user.organizationName=My Company` is invalid
- **Format**: must be comma-separated key=value pairs: `key1=value1,key2=value2`
- **Allowed characters**: only US-ASCII characters excluding control characters, whitespace, double quotes, commas, semicolons, and backslashes
- **Special characters**: characters outside the allowed range must be percent-encoded

For a value that would need a space, use underscores or camelCase instead. The following examples set `org.name` with each form:

```
export
OTEL_RESOURCE_ATTRIBUTES="org.name=Johns_Organization"
export
OTEL_RESOURCE_ATTRIBUTES="org.name=JohnsOrganization"
```

You can percent-encode any character, not only the excluded ones. This example encodes both the space and the apostrophe:

```
export
OTEL_RESOURCE_ATTRIBUTES="org.name=John%27s%20Organizatio
n"
```

Wrapping values in quotes doesn't escape spaces. For example, `org.name="My Company"` results in the literal value `"My Company"` with the quotes included, not `My Company`.

Example configurations

Set these environment variables before running `claude`. Each block shows a complete configuration for a different exporter or deployment scenario:

```
# Console debugging (1-second intervals)
export CLAUDE_CODE_ENABLE_TELEMETRY=1
export OTEL_METRICS_EXPORTER=console
export OTEL_METRIC_EXPORT_INTERVAL=1000

# OTLP/gRPC
export CLAUDE_CODE_ENABLE_TELEMETRY=1
export OTEL_METRICS_EXPORTER=otlp
export OTEL_EXPORTER_OTLP_PROTOCOL=grpc
export OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317

# Prometheus
export CLAUDE_CODE_ENABLE_TELEMETRY=1
export OTEL_METRICS_EXPORTER=prometheus

# Multiple exporters
export CLAUDE_CODE_ENABLE_TELEMETRY=1
export OTEL_METRICS_EXPORTER=console,otlp
export OTEL_EXPORTER_OTLP_PROTOCOL=http/json

# Different endpoints/backends for metrics and logs
export CLAUDE_CODE_ENABLE_TELEMETRY=1
export OTEL_METRICS_EXPORTER=otlp
export OTEL_LOGS_EXPORTER=otlp
export OTEL_EXPORTER_OTLP_METRICS_PROTOCOL=http/protobuf
export
OTEL_EXPORTER_OTLP_METRICS_ENDPOINT=http://metrics.example.com:43
18
export OTEL_EXPORTER_OTLP_LOGS_PROTOCOL=grpc
export
OTEL_EXPORTER_OTLP_LOGS_ENDPOINT=http://logs.example.com:4317

# Metrics only (no events/logs)
export CLAUDE_CODE_ENABLE_TELEMETRY=1
export OTEL_METRICS_EXPORTER=otlp
export OTEL_EXPORTER_OTLP_PROTOCOL=grpc
export OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317

# Events/logs only (no metrics)
export CLAUDE_CODE_ENABLE_TELEMETRY=1
export OTEL_LOGS_EXPORTER=otlp
export OTEL_EXPORTER_OTLP_PROTOCOL=grpc
export OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317
```

Available metrics and events

Standard attributes

All metrics and events share these standard attributes:

Attribute	Description	Controlled By
`session.id`	Unique session identifier	`OTEL_METRICS_INCLUDE_SESSION_ID` (default: true)
`app.version`	Current Claude Code version	`OTEL_METRICS_INCLUDE_VERSION` (default: false)
`app.entrypoint`	How the session was launched, such as `cli`, `sdk-cli`, `sdk-ts`, `sdk-py`, or `claude-vscode`	`OTEL_METRICS_INCLUDE_ENTRYPOINT` (default: false)
`organization.id`	Organization UUID (when authenticated)	Always included when available
`user.account_uuid`	Account UUID (when authenticated)	`OTEL_METRICS_INCLUDE_ACCOUNT_UUID` (default: true)
`user.account_id`	Account ID in tagged format matching Anthropic admin APIs (when authenticated), such as `user_01BWBeN28...`	`OTEL_METRICS_INCLUDE_ACCOUNT_UUID` (default: true)
`user.id`	Random anonymous identifier generated on first run and persisted in `~/.claude.json`. It contains no personal information and is not derived from your Claude account. Deleting the file produces a new unrelated value on next run.	Always included
`user.email`	User email address (when authenticated via OAuth)	Always included when available
`terminal.type`	Terminal type, such as `iTerm.app`, `vscode`, `cursor`, or `tmux`	Always included when detected
Keys from `OTEL_RESOURCE_ATTRIBUTES`	Custom attributes you set, such as `department` or `team.id`. See **Multi-team organization support**	`OTEL_METRICS_INCLUDE_RESOURCE_ATTRIBUTES` (default: true)

Events additionally include the following attributes. These are never attached to metrics because they would cause unbounded cardinality:

- `prompt.id`: UUID correlating a user prompt with all subsequent events until the next prompt. See **Event correlation attributes**.
- `workspace.host_paths`: host workspace directories selected in the desktop app, as a string array

Metrics

Claude Code exports the following metrics:

Metric Name	Description	Unit
`claude_code.session.count`	Count of CLI sessions started	count
`claude_code.lines_of_code.count`	Count of lines of code modified	count
`claude_code.pull_request.count`	Number of pull requests created	count
`claude_code.commit.count`	Number of git commits created	count
`claude_code.cost.usage`	Cost of the Claude Code session	USD
`claude_code.token.usage`	Number of tokens used	tokens
`claude_code.code_edit_tool.decision`	Count of code editing tool permission decisions	count
`claude_code.active_time.total`	Total active time in seconds	s

Metric details

Each metric includes the standard attributes listed above. Metrics with additional context-specific attributes are noted below.

Session counter

Incremented at the start of each session.

Attributes:

- All **standard attributes**
- `start_type`: How the session was started. One of `"fresh"`, `"resume"`, `"continue"`, or `"agents_view"`. The `"agents_view"` value identifies the `claude agents` dashboard process, a user-launched local UI rather than a conversational session. Filter on this value to separate UI process launches from conversational sessions in your dashboards.

Lines of code counter

Incremented when code is added or removed.

Attributes:

- All **standard attributes**
- `type`: (`"added"`, `"removed"`)
- `model`: Model identifier for the model that made the change (for example, “claude-sonnet-4-6”). Requires Claude Code v2.1.172 or later

Pull request counter

Incremented when Claude Code creates a pull request or merge request through a shell command or an MCP tool.

Attributes:

- All **standard attributes**

Commit counter

Incremented when creating git commits via Claude Code.

Attributes:

- All **standard attributes**

Cost counter

Incremented after each API request.

Attributes:

- All **standard attributes**
- `model`: Model identifier (for example, “claude-sonnet-4-6”)
- `query_source`: Category of the subsystem that issued the request. One of `"main"`, `"subagent"`, or `"auxiliary"`
- `speed`: `"fast"` when the request used fast mode. Absent otherwise
- `effort`: **Effort level** applied to the request: `"low"`, `"medium"`, `"high"`, `"xhigh"`, or `"max"`. Absent when the model doesn’t support effort.
- `agent.name`: Subagent type that issued the request. Built-in agent names and agents from official-marketplace plugins appear verbatim. Other user-defined agent names are replaced with `"custom"`. Absent when the request was not issued by a named subagent type.
- `skill.name`: Skill active for the request, set by the Skill tool, a `/` command, or inherited by a

spawned subagent. Built-in, bundled, user-defined, and official-marketplace plugin skill names appear verbatim. Third-party plugin skill names are replaced with `"third-party"`. Absent when no skill is active.

- `plugin.name`: Owning plugin when the active skill or subagent is provided by a plugin. Official-marketplace plugin names appear verbatim. Third-party plugin names are replaced with `"third-party"`. Absent when neither the skill nor the subagent has an owning plugin.
- `marketplace.name`: Marketplace the owning plugin was installed from. Only emitted for official-marketplace plugins. Absent otherwise.
- `mcp_server.name`: MCP server whose tool ran in the turn that produced this request. Built-in, claude.ai-proxied, and official-registry server names appear verbatim. User-configured server names are replaced with `"custom"`. Absent when no MCP tool ran.
- `mcp_tool.name`: MCP tool that ran in the turn that produced this request, with the same redaction as `mcp_server.name`. Absent when no MCP tool ran.

Token counter

Incremented after each API request.

Attributes:

- All **standard attributes**
- `type`: (`"input"`, `"output"`, `"cacheRead"`, `"cacheCreation"`)
- `model`: Model identifier (for example, “claude-sonnet-4-6”)
- `query_source`: Category of the subsystem that issued the request. One of `"main"`, `"subagent"`, or `"auxiliary"`
- `speed`: `"fast"` when the request used fast mode. Absent otherwise
- `effort`: **Effort level** applied to the request. See **Cost counter** for details.
- `agent.name`, `skill.name`, `plugin.name`, `marketplace.name`, `mcp_server.name`, `mcp_tool.name`: Skill, plugin, agent, and MCP attribution for the request. See **Cost counter** for definitions and redaction behavior.

Code edit tool decision counter

Incremented when user accepts or rejects Edit, Write, or NotebookEdit tool usage.

Attributes:

- All **standard attributes**
- `tool_name`: Tool name (`"Edit"`, `"Write"`, `"NotebookEdit"`)
- `decision`: User decision (`"accept"`, `"reject"`)
- `source`: Where the decision came from. One of `"config"`, `"hook"`, `"user_permanent"`, `"user_temporary"`, `"user_abort"`, or `"user_reject"`. See the **Tool decision event** for what each value means.
- `language`: Programming language of the edited file, such as `"TypeScript"`, `"Python"`, `"JavaScript"`, or `"Markdown"`. Returns `"unknown"` for unrecognized file extensions.

Active time counter

Tracks actual time spent actively using Claude Code, excluding idle time. This metric is incremented during user interactions, such as typing and reading responses, and during CLI processing, such as tool execution and AI response generation.

Attributes:

- All **standard attributes**
- `type`: `"user"` for keyboard interactions, `"cli"` for tool execution and AI responses

Events

Claude Code exports the following events via OpenTelemetry logs/events (when `OTEL_LOGS_EXPORTER` is configured):

Event correlation attributes

When a user submits a prompt, Claude Code may make multiple API calls and run several tools. The `prompt.id` attribute lets you tie all of those events back to the single prompt that triggered them.

Attribute	Description
`prompt.id`	UUID v4 identifier linking all events produced while processing a single user prompt

To trace all activity triggered by a single prompt, filter your events by a specific `prompt.id` value. This returns the user_prompt event, any api_request events, and any tool_result events that occurred while processing that prompt.

`prompt.id` is intentionally excluded from metrics because each prompt generates a unique ID, which would create an ever-growing number of time series. Use it for event-level analysis and audit trails only.

User prompt event

Logged when a user submits a prompt.

Event Name: `claude_code.user_prompt`

Attributes:

- All **standard attributes**
- `event.name`: `"user_prompt"`
- `event.timestamp`: ISO 8601 timestamp
- `event.sequence`: monotonically increasing counter for ordering events within a session
- `prompt_length`: Length of the prompt
- `prompt`: Prompt content. Redacted by default. Set `OTEL_LOG_USER_PROMPTS=1` to include it
- `command_name`: Command name when the prompt invokes one. Built-in and bundled command names such as `compact` or `debug` are emitted as-is; aliases such as `reset` emit as typed rather than the canonical name. Custom, plugin, and MCP command names collapse to `custom` or `mcp` unless `OTEL_LOG_TOOL_DETAILS=1` is set
- `command_source`: Origin of the command when present: `builtin`, `custom`, or `mcp`. Plugin-provided commands report as `custom`

Tool result event

Logged when a tool completes execution. Not emitted if the tool call was rejected; see the **Tool decision event** for rejections.

Event Name: `claude_code.tool_result`

Attributes:

- All **standard attributes**
- `event.name`: `"tool_result"`
- `event.timestamp`: ISO 8601 timestamp
- `event.sequence`: monotonically increasing counter for ordering events within a session

- `tool_name`: Name of the tool
- `tool_use_id`: Unique identifier for this tool invocation. Matches the `tool_use_id` passed to hooks, allowing correlation between OTel events and hook-captured data.
- `success`: `"true"` or `"false"`
- `duration_ms`: Execution time in milliseconds
- `error_type`: Error category string when the tool failed, such as `"Error:ENOENT"` or `"ShellError"`
- `error` (when `OTEL_LOG_TOOL_DETAILS=1`): Full error message when the tool failed
- `decision_type`: Always `"accept"`, since this event is only emitted after the tool runs. Rejected calls don't produce a tool result
- `decision_source`: Where the permission decision came from. One of `"config"`, `"hook"`, `"user_permanent"`, or `"user_temporary"`. See the **Tool decision event** for what each value means. The reject-only sources `"user_abort"` and `"user_reject"` never appear on this event.
- `tool_input_size_bytes`: Size of the JSON-serialized tool input in bytes
- `tool_result_size_bytes`: Size of the tool result in bytes
- `mcp_server_scope`: MCP server scope identifier (for MCP tools)
- `tool_parameters` (when `OTEL_LOG_TOOL_DETAILS=1`): JSON string containing tool-specific parameters:
  - For Bash tool: includes `bash_command`, `full_command`, `timeout`, `description`, `dangerouslyDisableSandbox`, and `git_commit_id` (the commit SHA, when a `git commit` command succeeds)
  - For WorkspaceBash tool: includes `bash_command`, `full_command`, `timeout`
  - For MCP tools: includes `mcp_server_name`, `mcp_tool_name`
  - For Skill tool: includes `skill_name`
  - For Agent tool or legacy Task tool: includes `subagent_type`
- `tool_input` (when `OTEL_LOG_TOOL_DETAILS=1`): JSON-serialized tool arguments. Individual values over 512 characters are truncated, and the full payload is bounded to ~4 K characters. Applies to all tools including MCP tools.

API request event

Logged for each API request to Claude.

Event Name: `claude_code.api_request`

Attributes:

- All **standard attributes**
- `event.name` : `"api_request"`
- `event.timestamp` : ISO 8601 timestamp
- `event.sequence` : monotonically increasing counter for ordering events within a session
- `model` : Model used (for example, “claude-sonnet-4-6”)
- `cost_usd` : Estimated cost in USD
- `duration_ms` : Request duration in milliseconds
- `input_tokens` : Number of input tokens
- `output_tokens` : Number of output tokens
- `cache_read_tokens` : Number of tokens read from cache
- `cache_creation_tokens` : Number of tokens used for cache creation
- `request_id` : Anthropic API request ID from the response’s `request-id` header, such as `"req_011..."`. Present only when the API returns one.
- `speed` : `"fast"` or `"normal"`, indicating whether fast mode was active
- `query_source` : Subsystem that issued the request, such as `"repl_main_thread"`, `"compact"`, or a subagent name
- `effort` : **Effort level** applied to the request: `"low"`, `"medium"`, `"high"`, `"xhigh"`, or `"max"`. Absent when the model doesn’t support effort.
- `agent.name`, `skill.name`, `plugin.name`, `marketplace.name`, `mcp_server.name`, `mcp_tool.name` : Skill, plugin, agent, and MCP attribution for the request. See **Cost counter** for definitions and redaction behavior.

API error event

Logged when an API request to Claude fails.

Event Name: `claude_code.api_error`

Attributes:

- All **standard attributes**
- `event.name` : `"api_error"`
- `event.timestamp` : ISO 8601 timestamp

- `event.sequence` : monotonically increasing counter for ordering events within a session
- `model` : Model used (for example, “claude-sonnet-4-6”)
- `error` : Error message
- `status_code` : HTTP status code as a number. Absent for non-HTTP errors such as connection failures.
- `duration_ms` : Request duration in milliseconds
- `attempt` : Total number of attempts made, including the initial request (`1` means no retries occurred)
- `request_id` : Anthropic API request ID from the response’s `request-id` header, such as `"req_011..."` . Present only when the API returns one.
- `speed` : `"fast"` or `"normal"` , indicating whether fast mode was active
- `query_source` : Subsystem that issued the request, such as `"repl_main_thread"` , `"compact"` , or a subagent name
- `effort` : **Effort level** applied to the request. Absent when the model doesn’t support effort.
- `agent.name` , `skill.name` , `plugin.name` , `marketplace.name` , `mcp_server.name` , `mcp_tool.name` : Skill, plugin, agent, and MCP attribution for the request. See **Cost counter** for definitions and redaction behavior.

API refusal event

Logged when an API request returns `stop_reason: "refusal"` . Refusals arrive on a successful response stream rather than as an HTTP error, so the `api_error` event doesn’t fire for them. This event lets you track refusal frequency and group refusals by the same attributes as `api_request` and `api_error` .

Event Name: `claude_code.api_refusal`

Attributes:

- All **standard attributes**
- `event.name` : `"api_refusal"`
- `event.timestamp` : ISO 8601 timestamp
- `event.sequence` : monotonically increasing counter for ordering events within a session
- `model` : Model identifier from the request
- `request_id` : Anthropic API request ID from the response’s `request-id` header, such as

`"req_011..."`. Present only when the API returns one.

- `query_source`: Subsystem that issued the request, such as `"repl_main_thread"`, `"compact"`, or a subagent name. See `api_request` for definitions.
- `speed`: Either `"fast"` when **Fast mode** is active, or `"normal"`
- `attempt`: Retry attempt number. The first attempt is `1`.
- `effort`: **Effort level** applied to the request. Absent when the model doesn't support effort.
- `server_fallback_hop`: `true` when the API's server-side model fallback already retried this refusal on a different model, so the user did not see this particular refusal. `false` when the request ended in a refusal. A single turn can emit both a `true` hop event and a later `false` final event when the fallback model also refuses.
- `has_category`: `true` when the API response carried a `stop_details.category` of `"cyber"`, `"bio"`, `"frontier_llm"`, or `"reasoning_extraction"`. `false` when the response carried no category or a value outside that set. Absent when `server_fallback_hop` is `true`, because hop blocks don't carry `stop_details`.
- `has_explanation`: `true` when the API response carried a `stop_details.explanation`, otherwise `false`. Absent when `server_fallback_hop` is `true`.
- `category`: The `stop_details.category` value from the API response. One of `"cyber"`, `"bio"`, `"frontier_llm"`, or `"reasoning_extraction"`. Only present when `OTEL_LOG_TOOL_DETAILS=1` is set and `has_category` is `true`.
- `agent.name`, `skill.name`, `plugin.name`, `marketplace.name`, `mcp_server.name`, `mcp_tool.name`: Skill, plugin, agent, and MCP attribution for the request. See **Cost counter** for definitions and redaction behavior.

API request body event

Logged for each API request attempt when `OTEL_LOG_RAW_API_BODIES` is set. One event is emitted per attempt, so retries with adjusted parameters each produce their own event.

Event Name: `claude_code.api_request_body`

Attributes:

- All **standard attributes**
- `event.name`: `"api_request_body"`
- `event.timestamp`: ISO 8601 timestamp
- `event.sequence`: monotonically increasing counter for ordering events within a session

- `body` : JSON-serialized Messages API request parameters (system prompt, messages, tools, etc.), truncated at 60 KB. Extended-thinking content in prior assistant turns is redacted. Emitted only in inline mode (`OTEL_LOG_RAW_API_BODIES=1`).
- `body_ref` : Absolute path to a `<dir>/<uuid>.request.json` file containing the untruncated body. Emitted only in file mode (`OTEL_LOG_RAW_API_BODIES=file:<dir>`).
- `body_length` : Untruncated body length. UTF-8 bytes when `OTEL_LOG_RAW_API_BODIES=file:<dir>` , or UTF-16 code units when `=1`
- `body_truncated` : `"true"` when inline truncation occurred. Absent in file mode and when no truncation occurred.
- `model` : Model identifier from the request parameters
- `query_source` : Subsystem that issued the request (for example, `"compact"`)

API response body event

Logged for each successful API response when `OTEL_LOG_RAW_API_BODIES` is set.

Event Name: `claude_code.api_response_body`

Attributes:

- All **standard attributes**
- `event.name` : `"api_response_body"`
- `event.timestamp` : ISO 8601 timestamp
- `event.sequence` : monotonically increasing counter for ordering events within a session
- `body` : JSON-serialized Messages API response (id, content blocks, usage, stop reason), truncated at 60 KB. Extended-thinking content is redacted. Emitted only in inline mode (`OTEL_LOG_RAW_API_BODIES=1`).
- `body_ref` : Absolute path to a `<dir>/<request_id>.response.json` file containing the untruncated body. Emitted only in file mode (`OTEL_LOG_RAW_API_BODIES=file:<dir>`).
- `body_length` : Untruncated body length. UTF-8 bytes when `OTEL_LOG_RAW_API_BODIES=file:<dir>` , or UTF-16 code units when `=1`
- `body_truncated` : `"true"` when inline truncation occurred. Absent in file mode and when no truncation occurred.
- `model` : Model identifier
- `query_source` : Subsystem that issued the request

- `request_id`: Anthropic API request ID from the response's `request-id` header, such as `"req_011..."`. Present only when the API returns one.

Tool decision event

Logged when a tool permission decision is made (accept/reject).

Event Name: `claude_code.tool_decision`

Attributes:

- All **standard attributes**
- `event.name`: `"tool_decision"`
- `event.timestamp`: ISO 8601 timestamp
- `event.sequence`: monotonically increasing counter for ordering events within a session
- `tool_name`: Name of the tool (for example, "Read", "Edit", "Write", "NotebookEdit")
- `tool_use_id`: Unique identifier for this tool invocation. Matches the `tool_use_id` passed to hooks, allowing correlation between OTel events and hook-captured data.
- `decision`: Either `"accept"` or `"reject"`
- `source`: Where the decision came from:
  - `"config"`: Decided automatically without prompting, based on project settings, allow or deny rules in the user's personal settings, enterprise managed policy, `--allowedTools` or `--disallowedTools` flags, the active permission mode, a session-scoped grant from an earlier prompt in the same interactive CLI session, or because the tool is inherently safe. The event doesn't indicate which of these sources matched.
  - `"hook"`: A `PreToolUse` or `PermissionRequest` hook returned the decision.
  - `"user_permanent"`: Emitted when the user chose "Yes, and don't ask again for …" at a permission prompt, which saves an allow rule to their personal settings. In the interactive CLI this is emitted only for that choice itself; later calls that match the saved rule emit `"config"` instead. In Agent SDK or non-interactive `-p` sessions, both the initial choice and later rule matches emit `"user_permanent"`. Treated as an accept.
  - `"user_temporary"`: Emitted when the user chose "Yes" at a permission prompt for a one-time approval, or chose one of the "… during this session" options on a file edit or read prompt. In the interactive CLI this is emitted only for the choice itself; later calls allowed by that session-scoped grant emit `"config"` instead. In Agent SDK or non-interactive `-p` sessions, both the choice and later matches emit `"user_temporary"`. Treated as an accept.

- `"user_abort"`: Emitted when the user dismissed the permission prompt without answering. Treated as a reject.
- `"user_reject"`: Emitted when the user chose “No” when prompted. In the interactive CLI this is emitted only for that choice itself; calls that match a deny rule in the user’s personal settings emit `"config"` instead. In Agent SDK or non-interactive `-p` sessions, calls that match a deny rule in personal settings emit `"user_reject"`. Treated as a reject.

- `tool_parameters` (when `OTEL_LOG_TOOL_DETAILS=1`): JSON string containing tool-specific parameters. Same shape as the **Tool result event**, minus post-execution fields such as `git_commit_id`. Values may differ from `tool_result` for an accepted call if the permission decision rewrites the tool input via `updatedInput`. Use this attribute to see which command was rejected when `decision` is `"reject"`.
  - For Bash tool: includes `bash_command`, `full_command`, `timeout`, `description`, `dangerouslyDisableSandbox`
  - For WorkspaceBash tool: includes `bash_command`, `full_command`, `timeout`
  - For MCP tools: includes `mcp_server_name`, `mcp_tool_name`
  - For Skill tool: includes `skill_name`
  - For Agent tool or legacy Task tool: includes `subagent_type`

Permission mode changed event

Logged when the permission mode changes, for example from `Shift+Tab` cycling, exiting plan mode, or an auto mode gate check.

Event Name: `claude_code.permission_mode_changed`

Attributes:

- All **standard attributes**
- `event.name`: `"permission_mode_changed"`
- `event.timestamp`: ISO 8601 timestamp
- `event.sequence`: monotonically increasing counter for ordering events within a session
- `from_mode`: The previous permission mode, for example `"default"`, `"plan"`, `"acceptEdits"`, `"auto"`, or `"bypassPermissions"`
- `to_mode`: The new permission mode
- `trigger`: What caused the change. One of `"shift_tab"`, `"exit_plan_mode"`, `"auto_gate_denied"`, or `"auto_opt_in"`. Absent when the transition originates from the SDK

or bridge

Auth event

Logged when `/login` or `/logout` completes.

Event Name: `claude_code.auth`

Attributes:

- All **standard attributes**
- `event.name`: `"auth"`
- `event.timestamp`: ISO 8601 timestamp
- `event.sequence`: monotonically increasing counter for ordering events within a session
- `action`: `"login"` or `"logout"`
- `success`: `"true"` or `"false"`
- `auth_method`: Authentication method, such as `"oauth"`
- `error_category`: Categorical error kind when the action failed. The raw error message is never included
- `status_code`: HTTP status code as a string when the action failed with an HTTP error

MCP server connection event

Logged when an MCP server connects, disconnects, or fails to connect.

Event Name: `claude_code.mcp_server_connection`

Attributes:

- All **standard attributes**
- `event.name`: `"mcp_server_connection"`
- `event.timestamp`: ISO 8601 timestamp
- `event.sequence`: monotonically increasing counter for ordering events within a session
- `status`: `"connected"`, `"failed"`, or `"disconnected"`
- `transport_type`: Server transport, such as `"stdio"`, `"sse"`, or `"http"`
- `server_scope`: Scope the server is configured at, such as `"user"`, `"project"`, or `"local"`
- `duration_ms`: Connection attempt duration in milliseconds

- `error_code`: Error code when the connection failed
- `is_plugin`: `true` when the server is provided by a plugin, `false` otherwise
- `plugin_id_hash` (when `is_plugin` is `true`): Stable hash of the plugin name and marketplace, for grouping events by plugin without exposing the name
- `plugin.name` (when `is_plugin` is `true`): Name of the plugin that provides the server. For third-party plugins this is the literal string `"third-party"` unless `OTEL_LOG_TOOL_DETAILS=1`; this protects third-party plugin names from appearing in logs by default. Plugins from official Anthropic sources are always identified by name. The `plugin_id_hash` and `plugin.name` attributes flow to your own monitoring backend and are not sent to Anthropic
- `server_name` (when `OTEL_LOG_TOOL_DETAILS=1`): Configured server name
- `error` (when `OTEL_LOG_TOOL_DETAILS=1`): Full error message when the connection failed

Internal error event

Logged when Claude Code catches an unexpected internal error. Only the error class name and an errno-style code are recorded. The error message and stack trace are never included. This event is not emitted when running against Bedrock, Vertex, or Foundry, or when `DISABLE_ERROR_REPORTING` is set.

Event Name: `claude_code.internal_error`

Attributes:

- All **standard attributes**
- `event.name`: `"internal_error"`
- `event.timestamp`: ISO 8601 timestamp
- `event.sequence`: monotonically increasing counter for ordering events within a session
- `error_name`: Error class name, such as `"TypeError"` or `"SyntaxError"`
- `error_code`: Node.js errno code such as `"ENOENT"` when present on the error

Plugin installed event

Logged when a plugin finishes installing, from both the `claude plugin install` CLI command and the interactive `/plugin` UI.

Event Name: `claude_code.plugin_installed`

Attributes:

- All **standard attributes**
- `event.name` : `"plugin_installed"`
- `event.timestamp` : ISO 8601 timestamp
- `event.sequence` : monotonically increasing counter for ordering events within a session
- `marketplace.is_official` : `"true"` if the marketplace is an official Anthropic marketplace, `"false"` otherwise
- `install.trigger` : `"cli"` or `"ui"`
- `plugin.name` : Name of the installed plugin. For third-party marketplaces this is included only when `OTEL_LOG_TOOL_DETAILS=1`
- `plugin.version` : Plugin version when declared in the marketplace entry. For third-party marketplaces this is included only when `OTEL_LOG_TOOL_DETAILS=1`
- `marketplace.name` : Marketplace the plugin was installed from. For third-party marketplaces this is included only when `OTEL_LOG_TOOL_DETAILS=1`

Plugin loaded event

Logged once per enabled plugin at session start. Use this event to inventory which plugins are active across your fleet, as a complement to `plugin_installed` which records the install action itself.

Event Name: `claude_code.plugin_loaded`

Attributes:

- All **standard attributes**
- `event.name` : `"plugin_loaded"`
- `event.timestamp` : ISO 8601 timestamp
- `event.sequence` : monotonically increasing counter for ordering events within a session
- `plugin.name` : name of the plugin. For plugins outside the official marketplace and built-in bundle the value is `"third-party"` unless `OTEL_LOG_TOOL_DETAILS=1`
- `marketplace.name` : marketplace the plugin was installed from, when known. Redacted to `"third-party"` under the same condition as `plugin.name`
- `plugin.version` : version from the plugin manifest. Included only when the name is not redacted and the manifest declares a version
- `plugin.scope` : provenance category for the plugin: `"official"` , `"org"` , `"user-local"` , or `"default-bundle"`

- `enabled_via`: how the plugin came to be enabled: `"default-enable"`, `"org-policy"`, `"seed-mount"`, or `"user-install"`
- `plugin_id_hash`: deterministic hash of the plugin name and marketplace, sent only to your configured exporter. Lets you count how many distinct third-party plugins are loaded across your fleet without recording their names
- `has_hooks`: whether the plugin contributes hooks
- `has_mcp`: whether the plugin contributes MCP servers
- `host_owned_mcp`: `true` when the SDK host manages this plugin's MCP connections and Claude Code skipped reading the plugin's MCP server configuration, `false` otherwise. Requires Claude Code v2.1.172 or later
- `skill_path_count`: number of skill directories the plugin declares
- `command_path_count`: number of command directories the plugin declares
- `agent_path_count`: number of agent directories the plugin declares
- `safe_mode`: `"true"` when the session was started with `--safe-mode`, `"false"` otherwise. In safe mode this event reports configured inventory only; the plugin's commands, skills, hooks, and MCP servers don't load. Requires Claude Code v2.1.169 or later

Skill activated event

Logged when a skill is invoked, whether Claude calls it through the Skill tool or you run it as a `/` command.

Event Name: `claude_code.skill_activated`

Attributes:

- All **standard attributes**
- `event.name`: `"skill_activated"`
- `event.timestamp`: ISO 8601 timestamp
- `event.sequence`: monotonically increasing counter for ordering events within a session
- `skill.name`: Name of the skill. For user-defined and third-party plugin skills the value is the placeholder `"custom_skill"` unless `OTEL_LOG_TOOL_DETAILS=1`
- `invocation_trigger`: How the skill was triggered (`"user-slash"`, `"claude-proactive"`, or `"nested-skill"`)
- `skill.source`: Where the skill was loaded from (for example, `"bundled"`, `"userSettings"`, `"projectSettings"`, `"plugin"`)

- `skill.kind` : `"workflow"` when the skill is a workflow skill. Absent otherwise
- `plugin.name` (when `OTEL_LOG_TOOL_DETAILS=1` or the plugin is from an official marketplace): Name of the owning plugin when the skill is provided by a plugin
- `marketplace.name` (when `OTEL_LOG_TOOL_DETAILS=1` or the plugin is from an official marketplace): Marketplace the owning plugin was installed from, when the skill is provided by a plugin

At mention event

Logged when Claude Code resolves an `@` -mention in a prompt. Not every mention emits an event: early-exit paths such as permission denials, oversized files, PDF reference attachments, and directory listing failures return without logging.

Event Name: `claude_code.at_mention`

Attributes:

- All **standard attributes**
- `event.name` : `"at_mention"`
- `event.timestamp` : ISO 8601 timestamp
- `event.sequence` : monotonically increasing counter for ordering events within a session
- `mention_type` : Type of mention (`"file"` , `"directory"` , `"agent"` , `"mcp_resource"`)
- `success` : Whether the mention resolved successfully (`"true"` or `"false"`)

API retries exhausted event

Logged once when an API request fails after more than one attempt. Emitted alongside the final `api_error` event.

Event Name: `claude_code.api_retries_exhausted`

Attributes:

- All **standard attributes**
- `event.name` : `"api_retries_exhausted"`
- `event.timestamp` : ISO 8601 timestamp
- `event.sequence` : monotonically increasing counter for ordering events within a session
- `model` : Model used

- `error`: Final error message
- `status_code`: HTTP status code as a number. Absent for non-HTTP errors.
- `total_attempts`: Total number of attempts made
- `total_retry_duration_ms`: Total wall-clock time across all attempts
- `speed`: `"fast"` or `"normal"`

Hook registered event

Logged once per configured hook at session start. Use this event to inventory which hooks are active across your fleet, as a complement to the per-execution `hook_execution_start` and `hook_execution_complete` events.

Event Name: `claude_code.hook_registered`

Attributes:

- All **standard attributes**
- `event.name`: `"hook_registered"`
- `event.timestamp`: ISO 8601 timestamp
- `event.sequence`: monotonically increasing counter for ordering events within a session
- `hook_event`: hook event type, such as `"PreToolUse"` or `"PostToolUse"`
- `hook_type`: hook implementation type: `"command"`, `"prompt"`, `"mcp_tool"`, `"http"`, or `"agent"`
- `hook_source`: where the hook is defined: `"userSettings"`, `"projectSettings"`, `"localSettings"`, `"flagSettings"`, `"policySettings"`, or `"pluginHook"`
- `safe_mode`: `"true"` when the session was started with `--safe-mode`, `"false"` otherwise. Requires Claude Code v2.1.169 or later
- `hook_matcher` (when `OTEL_LOG_TOOL_DETAILS=1`): the matcher string from the hook configuration, when one is set
- `plugin.name` (when `hook_source` is `"pluginHook"`): name of the contributing plugin. For plugins outside the official marketplace and built-in bundle the value is `"third-party"` unless `OTEL_LOG_TOOL_DETAILS=1`
- `plugin_id_hash` (when `hook_source` is `"pluginHook"`): deterministic hash of the plugin name and marketplace, sent only to your configured exporter. Lets you count distinct contributing plugins without recording their names

Hook execution start event

Logged when one or more hooks begin executing for a hook event.

Event Name: `claude_code.hook_execution_start`

Attributes:

- All **standard attributes**
- `event.name` : `"hook_execution_start"`
- `event.timestamp` : ISO 8601 timestamp
- `event.sequence` : monotonically increasing counter for ordering events within a session
- `hook_event` : Hook event type, such as `"PreToolUse"` or `"PostToolUse"`
- `hook_name` : Full hook name including matcher, such as `"PreToolUse:Write"`
- `num_hooks` : Number of matching hook commands
- `managed_only` : `"true"` when only managed-policy hooks are permitted
- `hook_source` : `"policySettings"` or `"merged"`
- `safe_mode` : `"true"` when the session was started with `--safe-mode`, `"false"` otherwise. Requires Claude Code v2.1.169 or later
- `hook_definitions` : JSON-serialized hook configuration. Included only when both detailed beta tracing and `OTEL_LOG_TOOL_DETAILS=1` are enabled

Hook execution complete event

Logged when all hooks for a hook event have finished.

Event Name: `claude_code.hook_execution_complete`

Attributes:

- All **standard attributes**
- `event.name` : `"hook_execution_complete"`
- `event.timestamp` : ISO 8601 timestamp
- `event.sequence` : monotonically increasing counter for ordering events within a session
- `hook_event` : Hook event type
- `hook_name` : Full hook name including matcher
- `num_hooks` : Number of matching hook commands

- `num_success` : Count that completed successfully
- `num_blocking` : Count that returned a blocking decision
- `num_non_blocking_error` : Count that failed without blocking
- `num_cancelled` : Count cancelled before completion
- `total_duration_ms` : Wall-clock duration of all matching hooks
- `managed_only` : `"true"` when only managed-policy hooks are permitted
- `hook_source` : `"policySettings"` or `"merged"`
- `safe_mode` : `"true"` when the session was started with `--safe-mode`, `"false"` otherwise. Requires Claude Code v2.1.169 or later
- `hook_definitions` : JSON-serialized hook configuration. Included only when both detailed beta tracing and `OTEL_LOG_TOOL_DETAILS=1` are enabled

Hook plugin metrics event

Logged when an official-marketplace plugin hook emits per-invocation metrics. Only plugins installed from an official Anthropic marketplace can emit these. Third-party marketplace plugins and user-configured hooks don't emit to this event. Use this event to monitor plugin behavior such as finding rates, costs, and durations from your own observability stack.

Event Name: `claude_code.hook_plugin_metrics`

Attributes:

- All **standard attributes**
- `event.name` : `"hook_plugin_metrics"`
- `event.timestamp` : ISO 8601 timestamp
- `event.sequence` : monotonically increasing counter for ordering events within a session
- `plugin_id` : plugin identifier in `<name>@<marketplace>` form
- `hook_event` : hook event type that emitted the metrics
- Up to 20 plugin-emitted metric keys. Names match `^[a-z][a-z0-9_]{0,39}$` . Values are boolean or number.

Compaction event

Logged when conversation compaction completes.

Event Name: `claude_code.compaction`

Attributes:

- All **standard attributes**
- `event.name`: `"compaction"`
- `event.timestamp`: ISO 8601 timestamp
- `event.sequence`: monotonically increasing counter for ordering events within a session
- `trigger`: `"auto"` or `"manual"`
- `success`: `"true"` or `"false"`
- `duration_ms`: Compaction duration
- `pre_tokens`: Approximate token count before compaction
- `post_tokens`: Approximate token count after compaction
- `error`: Error message when compaction failed
- `precompute_reuse`: Only set when `trigger` is `"manual"`. Auto-compaction can prepare a summary in the background before the context window fills, and this attribute records whether `/compact` reused that prepared summary. `"hit"` means it was reused; `"miss_custom_instructions"`, `"miss_hook"`, and `"miss_not_ready"` give the reason a fresh summary was computed instead. Requires Claude Code v2.1.153 or later

Feedback survey event

Logged when a session quality survey is shown or answered. See **Session quality surveys** for what the surveys collect and how to control them.

Event Name: `claude_code.feedback_survey`

Attributes:

- All **standard attributes**
- `event.name`: `"feedback_survey"`
- `event.timestamp`: ISO 8601 timestamp
- `event.sequence`: monotonically increasing counter for ordering events within a session
- `event_type`: Survey lifecycle event, for example `"appeared"`, `"responded"`, or `"transcript_prompt_appeared"`
- `appearance_id`: Unique ID linking the events emitted for one survey instance

- `survey_type`: Which survey produced the event. `"session"` is the "How is Claude doing?" rating prompt
- `response`: The user's selection on `responded` events
- `enabled_via_override`: `true` when `CLAUDE_CODE_ENABLE_FEEDBACK_SURVEY_FOR_OTEL` is set. Emitted as a boolean, not a string. Present on `session` survey events. Filter on this attribute to confirm the override is applied across a fleet

Interpret metrics and events data

The exported metrics and events support a range of analyses:

Usage monitoring

Metric	Analysis Opportunity
`claude_code.token.usage`	Break down by `type` (input/output), user, team, model, `skill.name`, `plugin.name`, or `agent.name`
`claude_code.session.count`	Track adoption and engagement over time
`claude_code.lines_of_code.count`	Measure productivity by tracking code additions and removals, broken down by model
`claude_code.commit.count` & `claude_code.pull_request.count`	Understand impact on development workflows

Cost monitoring

The `claude_code.cost.usage` metric helps with:

- Tracking usage trends across teams or individuals
- Identifying high-usage sessions for optimization
- Attributing spend to specific skills, plugins, or subagent types via the `skill.name`, `plugin.name`, and `agent.name` attributes

> Cost metrics are approximations. For official billing data, refer to your API provider (Claude Console, Amazon Bedrock, or Google Cloud Vertex).

Alerting and segmentation

Common alerts to consider:

- Cost spikes
- Unusual token consumption
- High session volume from specific users

All metrics can be segmented by the **standard attributes**. The `model` attribute is available on `claude_code.token.usage`, `claude_code.cost.usage`, and from v2.1.172, `claude_code.lines_of_code.count`.

Per-model breakdowns of commits can only be approximated by joining against the token or cost metrics on `session.id`, since one session can span multiple models. Filter the token or cost side to rows where `query_source` is `"main"` so auxiliary and subagent requests don't attribute the session's commits to a model that didn't make them.

Detect retry exhaustion

Claude Code retries failed API requests internally and emits a single `claude_code.api_error` event only after it gives up, so the event itself is the terminal signal for that request. Intermediate retry attempts are not logged as separate events.

The `attempt` attribute on the event records how many attempts were made in total. `CLAUDE_CODE_MAX_RETRIES` defaults to 10 and is capped at 15. When the request exhausts all retries on a transient error, `attempt` equals one more than that effective limit: 11 by default, and never more than 16. A lower value indicates a non-retryable error such as a `400` response.

To distinguish a session that recovered from one that stalled, group events by `session.id` and check whether a later `api_request` event exists after the error.

Event analysis

The event data provides detailed insights into Claude Code interactions:

Tool usage patterns: analyze tool result events to identify:

- Most frequently used tools
- Tool success rates

- Average tool execution times
- Error patterns by tool type

Performance monitoring: track API request durations and tool execution times to identify performance bottlenecks.

Audit security events

OpenTelemetry events are the audit data source for Claude Code activity. Every event carries identity attributes that tie tool calls, MCP activity, and permission decisions back to the user who triggered them. The OTLP logs exporter can deliver these events to any Security Information and Event Management (SIEM) platform with an OTLP receiver, or to an OpenTelemetry Collector that forwards to your SIEM.

Attribute actions to users

The **standard attributes** on each event include the authenticated user's identity: `user.email`, `user.account_uuid`, `user.account_id`, and `organization.id` when signed in with a Claude account, plus the installation-scoped `user.id` and the per-session `session.id`.

MCP tool calls, Bash commands, and file edits are therefore attributed to the developer who started the session. Claude Code doesn't act under a separate service account; the identity recorded on each event is the developer's own Claude account.

When Claude Code authenticates with a direct API key, or against Bedrock, Vertex AI, or Microsoft Foundry, there is no Claude account in the session and only `user.id` and `session.id` are populated. In these deployments, attach user identity yourself with `OTEL_RESOURCE_ATTRIBUTES`, set per user through the **managed settings** file or a launch wrapper:

```
export
OTEL_RESOURCE_ATTRIBUTES="enduser.id=jdoe@example.com,enduser.dir
ectory_id=S-1-5-21-..."
```

Audit MCP activity

To capture MCP server activity with full call detail, enable the logs exporter and set `OTEL_LOG_TOOL_DETAILS=1`. Each MCP operation then produces structured events that carry the server name, tool name, and call arguments alongside the standard identity attributes:

Event	What it records for MCP
`mcp_server_connection`	Server connect, disconnect, and connection failure with `server_name`, `transport_type`, `server_scope`, and error detail
`tool_result`	Each MCP tool call with `tool_name` and `mcp_server_scope`, a `tool_parameters` payload containing `mcp_server_name` and `mcp_tool_name`, and a `tool_input` payload containing the call arguments
`tool_decision`	Whether the call was allowed or denied, whether the decision came from config, a hook, or the user, and a `tool_parameters` payload containing `mcp_server_name` and `mcp_tool_name`

Without `OTEL_LOG_TOOL_DETAILS`, these events drop the identifying detail:

- `tool_result`: keeps `tool_name` and `mcp_server_scope`, omits `mcp_server_name`, `mcp_tool_name`, and arguments
- `tool_decision`: keeps `tool_name`, omits `tool_parameters`
- `mcp_server_connection`: omits `server_name` and the error message, but keeps `is_plugin`, `plugin_id_hash`, and `plugin.name`, with non-Anthropic plugin names redacted to the literal `"third-party"`, so plugin-provided servers remain distinguishable without detailed logging

Map security questions to events

When building detection rules, look up the signal you want to monitor and query your backend for the corresponding event and attributes:

Signal	Event	Key attributes
Tool call allowed or denied, and by what	`tool_decision`	`decision`, `source`, `tool_name`, `tool_parameters`
Permission mode escalation	`permission_mode_changed`	`from_mode`, `to_mode`, `trigger`
Policy hook blocked an action	`hook_execution_complete`	`hook_event`, `num_blocking`
Login, logout, and authentication failure	`auth`	`action`, `success`, `error_category`
MCP server connect or failure	`mcp_server_connection`	`status`, `server_name`, `is_plugin`, `error_code`
Plugin installed and its source	`plugin_installed`	`plugin.name`, `marketplace.name`, `marketplace.is_official`
Commands run and files	`tool_result` (executed) or	`tool_parameters`; `tool_input`

Signal	Event	Key attributes
touched	`tool_decision` (rejected) with `OTEL_LOG_TOOL_DETAILS=1`	(`tool_result` only)

Claude Code emits the raw event stream only. Anomaly detection, baselining, correlation across sessions, and alerting are the responsibility of your SIEM or observability backend.

Send events to a SIEM

Point `OTEL_EXPORTER_OTLP_LOGS_ENDPOINT` at your SIEM's OTLP receiver, or at an OpenTelemetry Collector that forwards to your SIEM's native ingest API. The following managed-settings example exports events only, with full tool detail enabled for MCP and Bash auditing:

```
{
  "env": {
    "CLAUDE_CODE_ENABLE_TELEMETRY": "1",
    "OTEL_LOGS_EXPORTER": "otlp",
    "OTEL_LOG_TOOL_DETAILS": "1",
    "OTEL_EXPORTER_OTLP_LOGS_PROTOCOL": "http/protobuf",
    "OTEL_EXPORTER_OTLP_LOGS_ENDPOINT":
"https://siem.example.com:4318/v1/logs",
    "OTEL_EXPORTER_OTLP_HEADERS": "Authorization=Bearer your-
siem-token"
  }
}
```

Backend considerations

Your choice of metrics, logs, and traces backends determines the types of analyses you can perform:

For metrics

- **Time series databases (for example, Prometheus)**: Rate calculations, aggregated metrics
- **Columnar stores (for example, ClickHouse)**: Complex queries, unique user analysis
- **Full-featured observability platforms (for example, Honeycomb, Datadog, Grafana Cloud)**: Advanced querying, visualization, alerting

For events/logs

- **Log aggregation systems (for example, Elasticsearch, Loki)**: Full-text search, log analysis
- **Columnar stores (for example, ClickHouse)**: Structured event analysis
- **Full-featured observability platforms (for example, Honeycomb, Datadog, Grafana Cloud)**: Correlation between metrics and events

For traces

Choose a backend that supports distributed trace storage and span correlation:

- **Distributed tracing systems (for example, Jaeger, Zipkin, Grafana Tempo)**: Span visualization, request waterfalls, latency analysis
- **Full-featured observability platforms (for example, Honeycomb, Datadog, Grafana Cloud)**: Trace search and correlation with metrics and logs

For organizations requiring Daily/Weekly/Monthly Active User (DAU/WAU/MAU) metrics, consider backends that support efficient unique value queries.

Service information

All metrics and events are exported with the following resource attributes:

- `service.name`: `claude-code`
- `service.version`: Current Claude Code version
- `os.type`: Operating system type (for example, `linux`, `darwin`, `windows`)
- `os.version`: Operating system version string
- `host.arch`: Host architecture (for example, `amd64`, `arm64`)
- `wsl.version`: WSL version number (only present when running on Windows Subsystem for Linux)
- Meter Name: `com.anthropic.claude_code`

ROI measurement resources

For a comprehensive guide on measuring return on investment for Claude Code, including telemetry setup, cost analysis, productivity metrics, and automated reporting, see the **Claude Code ROI**

Measurement Guide. This repository provides ready-to-use Docker Compose configurations, Prometheus and OpenTelemetry setups, and templates for generating productivity reports integrated with tools like Linear.

Security and privacy

- OpenTelemetry export to your backend is opt-in and requires explicit configuration. For Anthropic's separate operational telemetry and how to disable it, see **Data usage**
- Raw file contents and code snippets are not included in metrics or events. Trace spans are a separate data path: see the `OTEL_LOG_TOOL_CONTENT` bullet below
- When authenticated via OAuth, `user.email` is included in telemetry attributes. If this is a concern for your organization, work with your telemetry backend to filter or redact this field
- User prompt content is not collected by default. Only prompt length is recorded. To include prompt content, set `OTEL_LOG_USER_PROMPTS=1`
- Tool input arguments and parameters are not logged by default. To include them, set `OTEL_LOG_TOOL_DETAILS=1`. This data is sent only to the OTEL endpoint you configure, never to Anthropic. Arguments may still contain sensitive values, so configure your telemetry backend to filter or redact these attributes as needed. When enabled:
  - `tool_result` and `tool_decision` events include a `tool_parameters` attribute with Bash commands, MCP server and tool names, and skill names. Fields such as `full_command` are emitted untruncated
  - `tool_result` events additionally include a `tool_input` attribute with file paths, URLs, search patterns, and other arguments. Individual values over 512 characters are truncated and the total is bounded to ~4 K characters
  - `user_prompt` events include the verbatim `command_name` for custom, plugin, and MCP commands
  - Trace spans include the same `tool_input` attribute and input-derived attributes such as `file_path`, with the same truncation as `tool_input`
- Tool input and output content is not logged in trace spans by default. To include it, set `OTEL_LOG_TOOL_CONTENT=1`. When enabled, span events include full tool input and output content truncated at 60 KB per span. This can include raw file contents from Read tool results and Bash command output. Configure your telemetry backend to filter or redact these attributes as needed
- Raw Anthropic Messages API request and response bodies are not logged by default. To include them, set `OTEL_LOG_RAW_API_BODIES`. With `=1`, each API call emits `api_request_body` and

`api_response_body` log events whose `body` attribute is the JSON-serialized payload, truncated at 60 KB. With `=file:<dir>`, untruncated bodies are written to `.request.json` and `.response.json` files under that directory and the events carry a `body_ref` path instead of the inline body. Ship the directory with a log collector or sidecar rather than through the telemetry stream. In both modes, bodies contain the full conversation history (system prompt, every prior user and assistant turn, tool results), so enabling this implies consent to everything the other `OTEL_LOG_*` content flags would reveal. Claude's extended-thinking content is always redacted from these bodies regardless of other settings

Monitor Claude Code on Amazon Bedrock

For detailed Claude Code usage monitoring guidance for Amazon Bedrock, see **Claude Code Monitoring Implementation (Bedrock)**.